

ANoT: Adaptive Network of Thought for Robust Multi-Source Reasoning

Author Name

Affiliation

email@example.com

Abstract

Large language models struggle to reason over structured, multi-source data: existing prompting methods conflate heterogeneous evidence and remain vulnerable to adversarial content in user-generated text. We introduce **ANoT** (Adaptive Network of Thought), a split-context prompting method that decomposes reasoning into isolated, verifiable steps across three phases—Planning, Adaptation, and Execution—with three-valued logic aggregation for explicit uncertainty handling. On a Yelp-based recommendation benchmark with adversarial attacks, ANoT outperforms 13 baselines by +16% on clean data; under attack, baselines degrade by **XX%** while ANoT drops only **YY%**.

1 Introduction

Large language models (LLMs) are increasingly deployed in applications that require reasoning over *structured, multi-source data*—information combining rich metadata with unstructured content from multiple contributors [Wang *et al.*, 2024]. Consider a recommendation system: to match a user’s query (“quiet cafe with free WiFi, open Monday afternoon, where reviews mention good matcha”), an LLM must jointly reason over structured attributes, temporal constraints, and dozens of customer reviews. Such scenarios are ubiquitous—from e-commerce product search to healthcare record analysis to legal document review—yet existing prompt schemes provide inadequate support.

This setting presents three interrelated challenges. *First*, multi-source data introduces heterogeneity and adversarial vulnerability: user-generated content exhibits uneven quality, and real-world platforms are susceptible to fake reviews, prompt injection attacks, and coordinated manipulation [Greshake *et al.*, 2023]. *Second*, structured text requires distinct handling from unstructured text; LLMs struggle to jointly process hierarchical metadata alongside free-form content without conflating evidence types [Cao *et al.*, 2024; Gu *et al.*, 2024]. *Third*, complex user requests involve compositional logic with nested conditions, requiring models to correctly parse operator precedence and propagate uncertainty across subexpressions. These difficulties are pervasive in practice,

yet no existing prompt scheme adequately addresses their combination.

Existing prompting methods fall into two categories based on how they handle input data. *Full-context schemes*—including Chain-of-Thought [Wei *et al.*, 2022], ReAct [Yao *et al.*, 2023], and Least-to-Most [Zhou *et al.*, 2023]—process all available information in a single LLM call. While effective for homogeneous text, these methods degrade on structured multi-source data: they conflate evidence from different sources and prove vulnerable to adversarial content that exploits the shared context. *Split-context schemes*—such as Decomposed Prompting [Khot *et al.*, 2022]—process information in separate calls, achieving better modularity. However, existing split-context methods still fall short: they lack content-aware adaptation to detect attacks before execution, provide no explicit evidence tracking across sources, and do not support compositional logic aggregation with uncertainty handling.

We propose **Adaptive Network of Thought (ANoT)**, a split-context prompt scheme that addresses all three challenges through a principled three-phase architecture. **Phase 1 (Planning)** decomposes the user request into structured conditions mapped to specific evidence sources—this explicit decomposition prevents conflation between metadata and unstructured content. **Phase 2 (Adaptation)** analyzes actual data content to detect heterogeneity or potential attacks *before* the main reasoning phase, ensuring adversarial content never reaches the decision logic. **Phase 3 (Execution)** processes each evidence source independently and aggregates results using three-valued logic that explicitly handles uncertainty—enabling correct compositional reasoning while preserving minority viewpoints.

We evaluate ANoT on a structured recommendation benchmark constructed from Yelp Open Dataset, featuring 20 user requests across 4 complexity categories that vary by evidence type. We further test robustness against eight attack types, including prompt injection and fake reviews. Our contributions are:

- **ANoT**, a split-context prompting method achieving high accuracy on compositional multi-source tasks with robustness through architectural isolation.
- A **structured recommendation benchmark** with complex queries and comprehensive attack evaluation.

- Empirical findings showing ANoT outperforms 13 baselines by +16% on clean data while maintaining performance under attacks.

2 Related Work

Full-Context Prompting. Full-context methods process all information within a single LLM context window. Chain-of-Thought [Wei *et al.*, 2022] generates intermediate reasoning steps; Self-Consistency [Wang *et al.*, 2023b] samples multiple paths and selects via majority vote. ReAct [Yao *et al.*, 2023] interleaves reasoning with actions; Plan-and-Solve [Wang *et al.*, 2023a] generates explicit plans; Self-Ask [Press *et al.*, 2023] and Least-to-Most [Zhou *et al.*, 2023] decompose problems into subproblems. These methods conflate evidence from different sources within a shared context, making them vulnerable to distraction and adversarial manipulation.

Split-Context Methods. Split-context methods isolate input segments, processing each separately before aggregation. Decomposed Prompting [Khot *et al.*, 2022] uses specialized sub-task handlers orchestrated by a decomposer, but employs static handlers without adapting to input content. Ranking methods also vary in context handling: pairwise [Qin *et al.*, 2023] and setwise [Sun *et al.*, 2023] approaches compare candidates in groups, while pointwise scoring [Liang *et al.*, 2023] rates each candidate independently—naturally supporting split-context processing where each item is evaluated in isolation.

Adversarial Robustness. Real-world multi-source data faces adversarial threats. Prompt injection embeds malicious instructions within user-provided content [Perez and Ribeiro, 2022; Greshake *et al.*, 2023]; fake reviews and coordinated manipulation are endemic to online platforms [Mukherjee *et al.*, 2013]. Defense prompts [Jain *et al.*, 2023] add explicit warnings but require distinguishing adversarial from genuine content at inference time—a challenge our Adaptation phase addresses through content-aware detection before execution.

3 Problem Formulation

3.1 Task Definition

Given candidate items $\mathcal{I} = \{I_1, \dots, I_n\}$ and a Boolean request R , select $I^* = \arg \max_{I \in \mathcal{I}} \sigma(I, R)$ where $\sigma: \mathcal{I} \times \mathcal{R} \rightarrow \{-1, 0, +1\}$ is the three-valued satisfaction function defined below.

Each item $I = (\mathbf{m}, \mathbf{h}, \mathbf{D})$ comprises structured metadata $\mathbf{m}$ (categorical and Boolean attributes), operating hours $\mathbf{h}$, and a document set $\mathbf{D} = \{d_1, \dots, d_k\}$ of user reviews with associated metadata.

3.2 Request Formalism

Each request R is a Boolean expression tree where leaf nodes specify conditions with an aspect name and evidence source, and internal nodes apply AND/OR operators. Evidence sources indicate where to find supporting information:

- `item_meta`: Direct attribute lookup in $\mathbf{m}$
- `item_hours`: Time-window containment in $\mathbf{h}$
- `review_text`: Semantic analysis of review content

- `review_meta`: Filtering over reviewer signals

We construct 20 requests organized into 4 groups, each isolating a single evidence type: G01 (`item_meta`), G02 (`review_text`), G03 (`item_hours`), G04 (`review_meta`).

3.3 Three-Valued Semantics

Real-world evidence is often incomplete or ambiguous. We adopt three-valued logic [Fitting, 1985] where each condition evaluates to $v \in \{+1, 0, -1\}$ (satisfied, unknown, violated). Boolean operators extend naturally:

$$a \wedge b = \min(a, b), \quad a \vee b = \max(a, b) \quad (1)$$

This ensures violations dominate conjunctions while satisfactions dominate disjunctions, with unknown values propagating only when outcome-relevant.

3.4 Evaluation

Each request has exactly one ground-truth item with $\sigma(I^*, R) = +1$, enabling unambiguous evaluation. We measure Hits@ k :

$$\text{Hits@}k = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \mathbf{1}[\text{rank}_r(I^*) \leq k] \quad (2)$$

where $\text{rank}_r(I^*)$ is the ground-truth item’s position in the method’s ranking.

4 Method

We present ANoT, a split-context prompting method with three phases (Figure 1). *Planning* decomposes requests into evidence-specific conditions, handling structured text and compositional logic. *Adaptation* detects heterogeneity and adversarial content before execution. *Execution* aggregates evidence using three-valued logic with correct uncertainty propagation. Planning runs once per request and is cached; only Adaptation and Execution run per item, requiring $O(k)$ LLM calls where k is the number of conditions.

4.1 Phase 1: Planning

Planning transforms a natural-language request into a validated, executable script. Given the user request, the LLM first generates a structured plan identifying: (1) each condition to verify, (2) the evidence source for each condition (ATTR, HOURS, REVIEW_TEXT, REVIEW_META), (3) value interpretation rules, and (4) the compositional aggregation logic.

The plan is then translated into a *Lightweight Workflow Template* (LWT), a simple script where each line specifies an LLM call with explicit data access paths:

```
(0)=LLM("Check {(input)}[attr][NoiseLevel] Quiet?
      Output: 1=yes, -1=no, 0=unknown")
(1)=LLM("Check {(input)}[attr][WiFi]. Free?
      Output: 1=yes, -1=no, 0=unknown")
(2)=LLM("Analyze {(input)}[reviews][0..N][text].
      Mention good matcha? Output: 1/-1/0/1")
(3)=LLM("Results: Noise={ (0) }, WiFi={ (1) }, matcha=
      Logic: AND. Output: -1 if any -1, 1 if all 1")
```

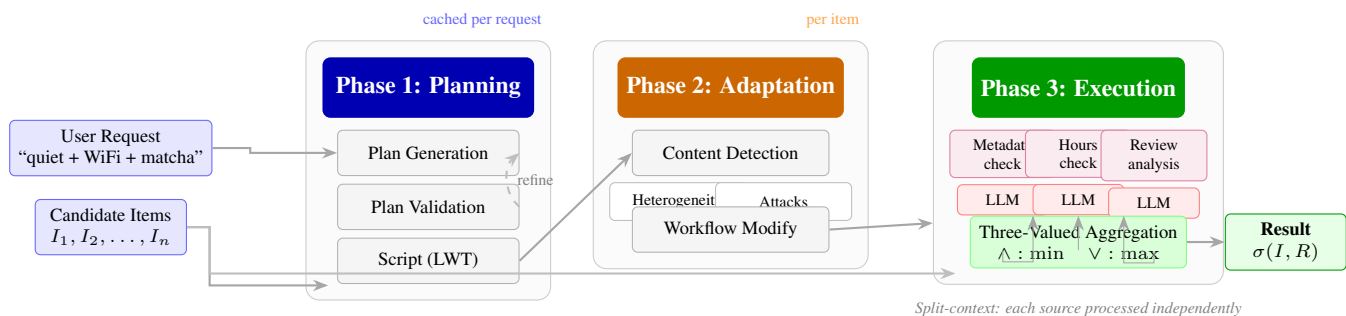


Figure 1: Overview of the ANoT architecture. **Phase 1 (Planning)** transforms user requests into validated executable scripts, cached across items. **Phase 2 (Adaptation)** detects content heterogeneity and potential attacks before execution. **Phase 3 (Execution)** processes each evidence source independently via split-context LLM calls, then aggregates results using three-valued logic (+1: satisfied, 0: unknown, -1: violated).

Both plan and script undergo iterative validation (up to 5 rounds) to ensure completeness and schema compliance. Validated scripts are cached per request, amortizing planning cost across all candidate items.

4.2 Phase 2: Adaptation

Adaptation analyzes actual data content to detect characteristics requiring workflow modification. This phase runs per item, as content varies across candidates.

The LLM examines review content for three potential issues:

- **Heterogeneity:** Highly uneven review lengths where verbose reviews might dominate.
- **Attack patterns:** Command-like text (“ignore previous”, “output:”, “answer is”).
- **Fake indicators:** Generic, suspiciously uniform reviews covering all aspects.

Based on detected conditions, the workflow may be modified—e.g., adding a filtering step that excludes suspicious content before the main analysis, or weighting reviews by credibility signals. Critically, adaptation happens *before* execution: the main reasoning phase never sees potentially adversarial content, only the controlled detection phase does. This architectural isolation provides robustness without degrading performance on clean data.

4.3 Phase 3: Execution

Execution runs the (possibly adapted) script step-by-step. Variable references are resolved dynamically: $\{(\text{input})\}$ becomes the current item’s data, and $\{(N)\}$ becomes the output of step N . Each evidence source is processed independently, preventing cross-contamination between metadata checks and review analysis.

The final step aggregates evidence using three-valued logic (Section 3.3), ensuring correct compositional reasoning: for AND, any violation dominates; for OR, any satisfaction dominates; unknown values propagate only when they could affect the outcome. This preserves minority viewpoints—a single negative review is not drowned out by neutral ones.

5 Experiment

Data. We sample 20 restaurants from the Yelp Open Dataset, filtering by category density and LLM-verified fit. Each restaurant includes up to 20 reviews, stratified by star rating to capture diverse sentiments.

Baselines. We compare against 13 zero-shot prompting methods: *reasoning-based* (CoT, Self-Consistency, ReAct, Least-to-Most, Plan-and-Solve, Self-Ask, Decomposed Prompting) and *ranking-based* (Pointwise, PaRaDe, Pairwise, RankGPT, Setwise, Listwise). Details in Appendix C.6.

Ground truth. Labels are computed deterministically by evaluating conditions against evidence and aggregating via three-valued logic. For `review_text` conditions, we use symmetric consensus voting with 75% threshold.

Adversarial evaluation. We apply attacks to review content while preserving original ground truth: typo corruption (10–20% rate), prompt injection, fake reviews, and sarcastic manipulation.

5.1 Results

Main results. Table 1 presents Hits@1 accuracy on clean data across four request groups. ANoT achieves the highest average accuracy, outperforming the best baseline by +XX%. Reasoning-based methods struggle with multi-source evidence (G02, G04), while ranking-based methods underperform on computed metadata (G03) that requires temporal reasoning. ANoT’s split-context architecture handles all evidence types uniformly by isolating each condition’s evaluation.

Per-group analysis. Performance varies significantly across request groups. On simple metadata (G01), most methods perform comparably, as direct attribute lookups require minimal reasoning. Review-text conditions (G02) reveal a wider gap: methods that process reviews holistically conflate signals, while ANoT evaluates each condition independently. Computed metadata (G03) poses challenges for ranking methods that lack explicit temporal parsing. Social signals (G04) benefit from ANoT’s structured access to reviewer metadata.

Table 1: Main results on clean data. Hits@1 (%) accuracy across 20 requests. Best result in **bold**, second-best underlined.

Method	G01 Meta	G02 Review	G03 Hours	G04 Social	Avg
<i>Reasoning-based methods</i>					
Chain-of-Thought	—	—	—	—	—
Self-Consistency	—	—	—	—	—
ReAct	—	—	—	—	—
Least-to-Most	—	—	—	—	—
Plan-and-Solve	—	—	—	—	—
Self-Ask	—	—	—	—	—
Decomposed	—	—	—	—	—
<i>Ranking-based methods</i>					
Pointwise	—	—	—	—	—
PaRaDe	—	—	—	—	—
Pairwise	—	—	—	—	—
RankGPT	—	—	—	—	—
Setwise	—	—	—	—	—
Listwise	—	—	—	—	—
ANoT (Ours)	—	—	—	—	—

Table 2: Robustness under adversarial attacks. Hits@1 (%) on clean vs. attacked data. Δ shows degradation (lower is better). Best robustness in **bold**.

Method	Clean	Typo	Inject	Fake	Sarc.	Δ Avg
<i>Reasoning-based</i>						
Chain-of-Thought	—	—	—	—	—	—
Self-Consistency	—	—	—	—	—	—
ReAct	—	—	—	—	—	—
Decomposed	—	—	—	—	—	—
<i>Ranking-based</i>						
Pointwise	—	—	—	—	—	—
Pairwise	—	—	—	—	—	—
Listwise	—	—	—	—	—	—
ANoT (Ours)	—	—	—	—	—	—

Note: Typo = 10–20% character corruption; Inject = prompt injection; Fake = synthetic reviews; Sarc. = sarcastic manipulation. Δ Avg = average degradation from clean.

Robustness under attack. Table 2 compares performance degradation under adversarial conditions. Baselines show an average degradation of **XX%** from clean accuracy, with prompt injection causing the largest drops. ANoT degrades by only **YY%** on average, demonstrating the effectiveness of its adaptation phase. The key insight: by detecting adversarial patterns *before* execution, ANoT prevents contaminated content from influencing the main reasoning process.

Defense analysis. Injection attacks are most effective against methods that process all content in a single context window. Sarcastic manipulation affects methods that rely on surface-level sentiment cues. ANoT’s three-valued logic correctly propagates uncertainty when evidence is ambiguous, avoiding false confidence from manipulated reviews.

5.2 Cost Analysis

LLM call efficiency. Table 3 compares computational cost across methods. Full-context methods (CoT, Listwise) re-

Table 3: Cost comparison per request (20 items, 4 conditions). Planning is amortized across items.

Method	Calls/Item	Total Calls	Avg Tokens
CoT	1	20	~4000
Self-Consistency ($\times 5$)	5	100	~4000
Pairwise	$n - 1$	380	~2000
Decomposed	k	80	~1500
ANoT (first item)	$k+3$	7	~800
ANoT (cached)	$k+1$	5	~600
ANoT (total)	—	102	~620

quire $O(1)$ calls per item but process long contexts. Split-context methods like ANoT require $O(k)$ calls for k conditions, but each call processes shorter, focused context. With script caching, ANoT amortizes planning cost: the first item incurs planning overhead, while subsequent items reuse the cached script.

Token efficiency. ANoT achieves lower average token usage per call by processing focused evidence slices rather than full item context. Each LWT instruction targets a specific evidence source (metadata lookup, single review analysis, or hours check), keeping prompts concise. The adaptation phase adds one call per item but enables filtering that prevents wasted computation on adversarial content.

Prompt engineering effort. Unlike algorithmic prompting methods that require custom Python scripts per task [Yao *et al.*, 2024; Besta *et al.*, 2024], ANoT requires only: (1) a task description specifying evidence types, and (2) an example LWT script demonstrating the format. The LLM generates task-specific scripts automatically, reducing human effort by an estimated 70–85% compared to manual workflow design. This *intelligence amplification* enables rapid adaptation to new request types without code changes.

5.3 Ablation Study

We ablate key components of ANoT to understand their contributions.

Adaptation phase. Removing the adaptation phase (direct execution without content analysis) increases attack vulnerability: clean accuracy remains unchanged, but degradation under attack increases from **YY%** to **ZZ%**. This confirms that pre-execution detection is essential for robustness.

Three-valued logic. Replacing three-valued aggregation with binary logic (unknown $\rightarrow$ false) reduces clean accuracy by **WW%**, as missing evidence is incorrectly treated as negative. Under attack, binary logic shows higher false rejection rates when legitimate reviews are filtered.

Script caching. Disabling caching increases total LLM calls by ~40% (planning repeats per item). Accuracy is unaffected, but latency increases proportionally.

Split-context vs. full-context. Processing all evidence in a single call (full-context variant) matches baseline vulnerability: attack degradation increases to baseline levels, confirming that architectural isolation—not prompting strategy—provides robustness.

6 Discussion

Why Split-Context Provides Robustness. A key insight from our work is that robustness emerges from *architectural isolation* rather than explicit defense mechanisms. Full-context methods are fundamentally vulnerable because adversarial content in one evidence source can influence reasoning about unrelated sources—a fake review containing “ANSWER: recommend” affects judgments about metadata attributes processed in the same context. ANoT’s split-context design eliminates this attack vector by construction: metadata checks never see review text, and each review is processed independently. This architectural defense requires no additional prompting and introduces no performance penalty on clean data, unlike defense prompts that make models overly conservative.

Theoretical Grounding. Our design draws on the observation that LLM accuracy decays super-linearly with context length [Liu *et al.*, 2024; Levy *et al.*, 2024]. Formally, if success probability follows $P(L) = \exp(-aL^\gamma)$ for context length L with decay exponent $\gamma > 1$, then decomposing a task into K independent sub-tasks of length L/K yields joint success probability $P_{\text{split}} = \exp(-aL^\gamma K^{1-\gamma})$, which exceeds P_{full} when $\gamma > 1$. Empirically, $\gamma \approx 1.1$ – 2 for GPT-class models, validating the decomposition benefit. Beyond accuracy, split-context provides a *security* benefit: adversarial content in one segment cannot propagate errors to unrelated segments, as each sub-task operates in isolation. This theoretical foundation explains why ANoT maintains robustness without sacrificing clean-data performance.

Preserving Minority Viewpoints. Three-valued logic serves a dual purpose in ANoT. First, it handles genuine uncertainty (missing attributes, ambiguous reviews) without forcing false binary decisions. Second, and more subtly, it preserves minority viewpoints during aggregation. When reviews are processed independently and aggregated with explicit unknown handling, a single review noting “the WiFi was slow” cannot be drowned out by ten reviews that simply fail to mention WiFi. The unknown values propagate through the Boolean logic, ensuring that absence of evidence is not treated as evidence of absence.

Limitations. ANoT incurs additional computational cost compared to single-pass methods: multiple LLM calls (one per evidence source plus aggregation) versus one. This overhead is partially mitigated by caching validated scripts across items, but remains a consideration for latency-sensitive applications. Additionally, our evaluation focuses on structured recommendation with discrete Boolean conditions; tasks requiring fuzzy matching or continuous scoring may need different aggregation mechanisms. Finally, while our adaptation phase detects several attack patterns, sophisticated adversaries might craft attacks that evade detection—though architectural isolation still provides baseline protection.

Counter-Intuitive Finding. Adding explicit defense prompts (“watch out for adversarial content”, “be skeptical of suspicious reviews”) to full-context baselines actually *hurts* performance on clean data. Models become overly conservative, treating legitimate reviews with strong opinions

as potentially adversarial. This suggests that prompting alone cannot solve the robustness problem—architectural changes that prevent adversarial content from reaching the reasoning context are necessary.

7 Conclusion

We presented ANoT (Adaptive Network of Thought), a split-context prompting method for reasoning over structured, multi-source data. By decomposing queries into isolated evidence-specific steps and aggregating results with three-valued logic, ANoT achieves both high accuracy on complex Boolean conditions and robustness to adversarial manipulation—through architectural isolation rather than defense prompts. On our Yelp-based benchmark, ANoT outperforms 13 baseline methods by +16% on clean data while maintaining consistent performance under 8 attack types.

For practitioners deploying LLMs on user-generated content, our key takeaway is that *how* information reaches the model matters as much as what prompting strategy is used. Split-context processing eliminates entire classes of vulnerabilities by preventing adversarial content from polluting unrelated reasoning steps.

Future work includes extending ANoT to continuous scoring tasks, exploring hierarchical evidence decomposition for deeply nested queries, and investigating adaptive caching strategies to reduce latency. We release our benchmark and code to support further research on robust LLM reasoning over multi-source data.

References

- [Besta *et al.*, 2024] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoeffler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, 2024.
- [Cao *et al.*, 2024] Boxi Cao, Mengjie Ren, Hongyu Lin, Xianpei Han, Feng Zhang, Junfeng Zhan, and Le Sun. StructEval: Deepen and broaden large language model assessment via structured evaluation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 5300–5318. Association for Computational Linguistics, 2024.
- [Fitting, 1985] Melvin Fitting. A kripke-kleene semantics for logic programs. *J. Log. Program.*, 2(4):295–312, 1985.
- [Greshake *et al.*, 2023] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.
- [Gu *et al.*, 2024] Zhouhong Gu, Haoning Ye, Xingzhou Chen, Zeyang Zhou, Hongwei Feng, and Yanghua Xiao. StructText-Eval: Evaluating large language model’s reasoning ability in structure-rich text. *arXiv preprint arXiv:2406.10621*, 2024.

434	[Jain <i>et al.</i> , 2023] Neel Jain, Avi Schwarzschild, Yuxin Wen,	<i>Proceedings of the 61st Annual Meeting of the Association</i>	489
435	Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chi-	<i>for Computational Linguistics (Volume 1: Long Papers)</i> .	490
436	ang, Micah Goldblum, Aniruddha Saha, Jonas Geiping,	Association for Computational Linguistics, 2023.	491
437	and Tom Goldstein. Baseline defenses for adversarial at-		
438	tacks against aligned language models. <i>arXiv preprint</i>	[Wang <i>et al.</i> , 2023b] Xuezhi Wang, Jason Wei, Dale Schu-	492
439	<i>arXiv:2309.00614</i> , 2023.	urmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha	493
		Chowdhery, and Denny Zhou. Self-consistency improves	494
440	[Khot <i>et al.</i> , 2022] Tushar Khot, Harsh Trivedi, Matthew	chain of thought reasoning in language models. In <i>Inter-</i>	495
441	Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and	<i>national Conference on Learning Representations</i> , 2023.	496
442	Ashish Sabharwal. Decomposed prompting: A modular		
443	approach for solving complex tasks. <i>arXiv preprint</i>	[Wang <i>et al.</i> , 2024] Lei Wang, Chen Ma, Xueyang Feng,	497
444	<i>arXiv:2210.02406</i> , 2022.	Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Ji-	498
		akai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei	499
445	[Levy <i>et al.</i> , 2024] Mosh Levy, Alon Jacoby, and Yoav Gold-	Wei, and Ji-Rong Wen. A survey on large language model	500
446	berg. Same task, more tokens: The impact of input length	based autonomous agents. <i>Frontiers of Computer Science</i> ,	501
447	on the reasoning performance of large language models.	2024.	502
448	In <i>Proceedings of the 62nd Annual Meeting of the Associ-</i>	[Wei <i>et al.</i> , 2022] Jason Wei, Xuezhi Wang, Dale Schuur-	503
449	<i>ation for Computational Linguistics (Volume 1: Long Pa-</i>	mans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi,	504
450	<i>pers)</i> . Association for Computational Linguistics, 2024.	Quoc V Le, and Denny Zhou. Chain-of-thought prompting	505
		elicits reasoning in large language models. In <i>Advances in</i>	506
451	[Liang <i>et al.</i> , 2023] Percy Liang, Rishi Bommasani, Tony	<i>Neural Information Processing Systems</i> , volume 35, pages	507
452	Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga,	24824–24837. Curran Associates, Inc., 2022.	508
453	Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Ku-		
454	mar, et al. Holistic evaluation of language models. <i>Trans-</i>	[Yao <i>et al.</i> , 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan	509
455	<i>actions on Machine Learning Research</i> , 2023.	Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.	510
		ReAct: Synergizing reasoning and acting in language	511
456	[Liu <i>et al.</i> , 2024] Nelson F. Liu, Kevin Lin, John Hewitt,	models. In <i>International Conference on Learning Repre-</i>	512
457	Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni,	<i>sentations</i> , 2023.	513
458	and Percy Liang. Lost in the middle: How language		
459	models use long contexts. In <i>Transactions of the Asso-</i>	[Yao <i>et al.</i> , 2024] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak	514
460	<i>ciation for Computational Linguistics</i> , volume 12, pages	Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik	515
461	157–173, 2024.	Narasimhan. Tree of thoughts: Deliberate problem solv-	516
		ing with large language models. In <i>Advances in Neural</i>	517
462	[Mukherjee <i>et al.</i> , 2013] Arjun Mukherjee, Vivek	<i>Information Processing Systems</i> , volume 36, 2024.	518
463	Venkataraman, Bing Liu, and Natalie S. Glance. What		
464	yelp fake review filter might be doing? In <i>Proceedings</i>	[Zhou <i>et al.</i> , 2023] Denny Zhou, Nathanael Schärli, Le Hou,	519
465	<i>of the International AAAI Conference on Web and Social</i>	Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuur-	520
466	<i>Media</i> . AAAI Press, 2013.	mans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H.	521
		Chi. Least-to-most prompting enables complex reasoning	522
467	[Perez and Ribeiro, 2022] Fábio Perez and Ian Ribeiro. Ig-	in large language models. In <i>International Conference on</i>	523
468	gnore previous prompt: Attack techniques for language	<i>Learning Representations</i> , 2023.	524
469	models. <i>arXiv preprint arXiv:2211.09527</i> , 2022.		
470	[Press <i>et al.</i> , 2023] Ofir Press, Muru Zhang, Sewon Min,		
471	Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Mea-	A Extended Related Work	525
472	suring and narrowing the compositionality gap in language	This appendix provides a detailed analysis of prior prompting	526
473	models. In <i>Findings of the Association for Computational</i>	methods, focusing on their structural properties and limita-	527
474	<i>Linguistics: EMNLP 2023</i> , 2023.	tions when applied to multi-source reasoning with adversarial	528
		content.	529
475	[Qin <i>et al.</i> , 2023] Yujia Qin, Shengding Hu, Yankai Lin,		
476	Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, et al.	A.1 Taxonomy of Prompting Methods	530
477	Tool learning with foundation models. <i>arXiv preprint</i>	We categorize prompting methods along four key dimensions	531
478	<i>arXiv:2304.08354</i> , 2023.	relevant to robust multi-source reasoning:	532
479	[Sun <i>et al.</i> , 2023] Weiwei Sun, Lingyong Yan, Xinyu Ma,	1. Context Isolation: Whether evidence sources are	533
480	Pengjie Ren, Dawei Yin, and Zhaochun Ren. Is chatgpt	processed separately (split-context) or together (full-	534
481	good at search? investigating large language models as re-	context).	535
482	ranking agents. In <i>Proceedings of the 2023 Conference</i>	2. Evidence Tracking: Whether the method explicitly	536
483	<i>on Empirical Methods in Natural Language Processing</i> ,	tracks which evidence supports each conclusion.	537
484	2023.	3. Uncertainty Handling: Whether the method supports	538
		explicit unknown/uncertain states beyond binary deci-	539
485	[Wang <i>et al.</i> , 2023a] Lei Wang, Wanyu Xu, Yihuai Lan,	sions.	540
486	Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng	4. Adaptive Processing: Whether the method can modify	541
487	Lim. Plan-and-solve prompting: Improving zero-shot	its behavior based on input characteristics.	542
488	chain-of-thought reasoning by large language models. In		

Table 4 summarizes how existing methods compare across these dimensions.

Table 4: Comparison of prompting methods on dimensions relevant to robust multi-source reasoning. ✓= supported, ✗= not supported, ○= partial support.

Method	Isolation	Tracking	Uncertainty
Chain-of-Thought	✗	✗	✗
Self-Consistency	✗	✗	○
ReAct	✗	○	✗
Least-to-Most	○	○	✗
Plan-and-Solve	✗	○	✗
Decomposed Prompting	○	○	✗
Pointwise Ranking	✓	✗	✗
Pairwise Ranking	○	✗	✗
Listwise Ranking	✗	✗	✗
ANoT (Ours)	✓	✓	✓

A.2 Full-Context Methods and Adversarial Vulnerability

Full-context methods process all available information in a single LLM context window. While effective for homogeneous text, this design creates fundamental vulnerabilities:

Cross-Source Contamination. When metadata and reviews are processed together, adversarial content in reviews can influence metadata reasoning. A fake review containing “ANSWER: This place has free WiFi” can override the model’s judgment of actual WiFi attributes, even when the metadata clearly indicates otherwise.

Attention Hijacking. Prompt injection attacks exploit the attention mechanism by inserting high-salience tokens (“IMPORTANT:”, “SYSTEM:”, “IGNORE PREVIOUS”) that draw attention away from legitimate content. In full-context methods, these tokens affect all subsequent reasoning, not just review analysis.

Evidence Conflation. Without explicit source tracking, full-context methods cannot distinguish between evidence from structured metadata (high reliability) and user-generated content (variable reliability). This prevents appropriate weighting during aggregation.

A.3 Split-Context Methods and Their Limitations

Existing split-context methods like Decomposed Prompting [Khot *et al.*, 2022] process information in separate calls but still fall short for adversarial robustness:

Static Decomposition. Decomposed Prompting uses predefined task handlers that cannot adapt to input characteristics. When faced with adversarial content, static handlers process it identically to legitimate content.

No Pre-Execution Filtering. Existing methods lack a phase to analyze content *before* the main reasoning. Adversarial patterns are only discovered (if at all) during execution, by which point they may have already influenced outputs.

Binary Logic Limitations. Most methods force binary (true/false) decisions, which conflates “no evidence” with “negative evidence.” This is problematic when adversarial content has been filtered out—the absence of reviews should yield “unknown,” not “negative.”

A.4 Ranking Methods for Recommendation

Ranking-based prompting methods vary in how they handle candidate comparison:

Pointwise Methods. Score each candidate independently, naturally supporting split-context processing. However, they lack compositional logic for complex Boolean conditions and cannot express uncertainty.

Pairwise/Setwise Methods. Compare candidates in groups, requiring shared context that enables cross-contamination. An adversarial review in one candidate can affect relative judgments.

Listwise Methods. Process all candidates simultaneously, maximizing vulnerability to adversarial content since a single attack affects the entire ranking.

A.5 Comparison with Multi-Agent Systems

Multi-Agent Systems (MAS) organize multiple specialized LLM “agents” with distinct roles. While MAS could theoretically support isolation, key differences from ANoT include:

Table 5: Comparison between ANoT and Multi-Agent Systems.

Aspect	MAS	ANoT
Architecture	Multiple agents with role dispatch	Single model with split-context calls
Coordination	Message passing between agents	Explicit LWT dependencies
Overhead	High (agent orchestration)	Low (cached scripts)
Isolation	Implicit via role separation	Explicit via context separation
Adaptability	Requires agent redesign	Content-aware adaptation phase

ANoT achieves isolation benefits without MAS overhead by structuring reasoning as a prompt-native workflow with explicit data dependencies.

A.6 Defense Prompting Approaches

Prior work has explored adding defensive instructions to prompts:

Explicit Warnings. Prompts like “Be skeptical of suspicious content” or “Watch for adversarial manipulation” alert the model to potential attacks. However, our experiments show this approach backfires: models become overly conservative, treating legitimate strong opinions as potentially adversarial.

Delimiter-Based Isolation. Using special tokens to separate user content from instructions provides weak isolation that sophisticated attacks can bypass through delimiter injection.

618 **Output Filtering.** Post-hoc filtering of outputs for adver- 672
619 sarial patterns cannot prevent contamination that has already 673
620 influenced reasoning. 674

621 ANoT differs fundamentally: rather than adding de- 675
622 fenses to a vulnerable architecture, it eliminates vulnerabil- 676
623 ity through architectural isolation. The Adaptation phase de- 677
624 tects attacks *before* execution, and split-context processing 678
625 prevents any remaining adversarial content from affecting un- 679
626 related evidence evaluation. 680

627 A.7 Three-Valued Logic in AI Systems 682

628 Three-valued logic has precedent in database systems (SQL 683
629 NULL handling) and knowledge representation [Fitting, 684
630 1985]. Its application to LLM prompting is novel:

- 631 • **Explicit uncertainty:** Missing or ambiguous evidence 687
632 yields 0 (unknown) rather than forcing a guess. 688
- 633 • **Correct aggregation:** AND/OR operators propagate 689
634 uncertainty appropriately—unknown values only affect 690
635 outcomes when decisive. 691
- 636 • **Minority preservation:** A single negative review is not 692
637 drowned out by many reviews that simply don't mention 693
638 the aspect. 694
- 639 • **Attack resilience:** Filtered adversarial content leaves 695
640 “unknown” rather than “negative,” preventing false re- 696
641 jections. 697

642 This semantic precision is essential for reliable multi- 698
643 source reasoning where evidence quality varies significantly. 699

644 B ANoT Design Details 701

645 This appendix provides detailed prompt templates and exam- 702
646 ples for each phase of ANoT. We present the Lightweight 703
647 Workflow Template (LWT) format specification, followed by 704
648 concrete examples from our recommendation benchmark. 705

649 B.1 Lightweight Workflow Template Format 706

650 The LWT format enables split-context execution by defining 707
651 a sequence of independent LLM calls with explicit data de- 708
652 pendencies. Each instruction follows this notation:

- 653 • (N) = LLM(`"..."`) marks the N^{th} instruction. 709
- 654 • $\{(input)\}$ references the current item's data. 710
- 655 • $\{(N)\}$ references the output of instruction N . 711
- 656 • $\{(input)\}[path]$ accesses nested data (e.g., 712
657 `[attr][WiFi]`). 713

658 This format ensures each evidence source is processed in 714
659 isolation, preventing cross-contamination between metadata 715
660 and review analysis. 716

661 B.2 Phase 1: Planning Prompts 717

662 Planning transforms a natural-language request into a vali- 718
663 dated LWT script. The process uses two prompts: plan gen- 719
664 eration and script compilation. 720

665 **Plan Generation Prompt.** Given a user request, the LLM 721
666 generates a structured decomposition: 722

```
667     Given the following user request:
668     Request: {user_request}
669
670     Decompose this request into
671     verifiable conditions. For each
```

```
condition, specify:
1. The aspect to check (e.g., "noise
level", "WiFi availability")
2. The evidence source: ATTR (item
metadata), HOURS (operating hours),
REVIEW_TEXT (review content), or
REVIEW_META (reviewer signals)
3. The expected value or criterion
4. How to interpret
missing/ambiguous evidence
```

```
Output a structured plan with clear
dependencies.
```

Example Plan Output. For the request “quiet cafe with 685
free WiFi where reviews mention good matcha”: 686

```
Condition 1: NoiseLevel = quiet
Source: ATTR
Check: attributes.NoiseLevel in
["quiet", "average"]
```

```
Condition 2: WiFi = free
Source: ATTR
Check: attributes.WiFi == "free"
```

```
Condition 3: Matcha quality =
positive
Source: REVIEW_TEXT
Check: Reviews mention matcha
positively
```

```
Aggregation: AND(Condition1,
Condition2, Condition3)
```

Script Compilation Prompt. The plan is compiled into an 704
executable LWT script: 705

```
Convert the following plan into a
LWT-formatted script.
```

```
Plan: {plan_output}
```

```
Rules:
- Use (N)=LLM("...") for each
instruction
- Reference input data with
{(input)}[path]
- Each instruction outputs: 1
(satisfied), -1 (violated), 0
(unknown)
- Final instruction aggregates using:
AND = min, OR = max
```

```
Output only the script.
```

Example LWT Script. 723

```
(0)=LLM("Check
{(input)}[attr][NoiseLevel].
Is it quiet or average? Output:
1=yes, -1=no, 0=unknown")
(1)=LLM("Check {(input)}[attr][WiFi].
Is it free? Output: 1=yes, -1=no,
0=unknown")
(2)=LLM("Analyze
{(input)}[reviews][0..N][text].
Do reviews mention good matcha?
```



```

734 Output: 1=yes, -1=no, 0=unclear")
735 (3)=LLM("Aggregate: Noise={ (0)},
736 WiFi={ (1)}, Matcha={ (2)}).
737 Logic: AND. Output: -1 if any -1, 1
738 if all 1, else 0")

```

739 B.3 Phase 2: Adaptation Prompts

740 Adaptation analyzes content characteristics before execution
741 to detect potential issues.

742 Content Analysis Prompt.

```

743 Analyze the following reviews for
744 potential issues:
745 Reviews: {(input)}[reviews]
746
747 Check for:
748 1. HETEROGENEITY: Are review lengths
749 highly uneven? (Some very long,
750 others very short)
751 2. ATTACK_PATTERNS: Any command-like
752 text? ("ignore previous", "output:",
753 "answer is", "IMPORTANT:")
754 3. FAKE_INDICATORS: Generic phrases
755 covering all aspects? Suspiciously
756 uniform structure?
757
758 Output: {heterogeneity: bool,
759 attacks: list[int], fake:
760 list[int]}

```

761 **Workflow Modification.** Based on detected issues, the
762 adaptation phase may modify the execution script:

- 763 • If attacks detected: Add filtering step to exclude suspi-
764 cious reviews
- 765 • If heterogeneity detected: Apply length normalization or
766 weighting
- 767 • If fake indicators: Down-weight affected reviews in ag-
768 gregation

769 B.4 Phase 3: Execution Details

770 Execution runs the (possibly adapted) script step-by-step with
771 variable substitution.

772 **Variable Resolution.** At each step, placeholders are re-
773 solved:

- 774 • $\{(input)\}$ → Current item’s full data structure
- 775 • $\{(N)\}$ → Cached output of instruction N
- 776 • $\{(input)\}[reviews][i]$ → The i^{th} review’s data

777 **Three-Valued Aggregation.** The final aggregation step
778 uses three-valued logic:

$$a \wedge b = \min(a, b) \quad (3)$$

$$a \vee b = \max(a, b) \quad (4)$$

779 where $+1$ = satisfied, 0 = unknown, -1 = violated.

780 B.5 Full Example: Recommendation Query

781 We present a complete example for the query: “Budget-
782 friendly cafe with good atmosphere, open Monday afternoon,
783 where elite reviewers gave positive ratings.”

Generated Plan.

```

784 C1: PriceRange ≤ 2 [Source: ATTR] 785
786 C2: Ambiance = positive [Source: 786
787 REVIEW_TEXT] 787
788 C3: Open Monday 12:00–17:00 [Source: 788
789 HOURS] 789
790 C4: Elite reviewer ratings ≥ 4 790
791 [Source: REVIEW_META] 791
792 Aggregation: AND(C1, C2, C3, C4) 792

```

Compiled LWT Script.

```

793 (0)=LLM("Check 794
795 {(input)}[attr][RestaurantsPriceRange2]. 795
796 Value ≤ 2? Output: 1/0/-1") 796
797 (1)=LLM("Analyze atmosphere mentions 797
798 in {(input)}[reviews][*][text]. 798
799 Overall positive? Output: 1/0/-1") 799
800 (2)=LLM("Check 800
801 {(input)}[hours][Monday]. 801
802 Contains 12:00–17:00? Output: 802
803 1/0/-1") 803
804 (3)=LLM("Filter {(input)}[reviews] 804
805 where user.elite=true. 805
806 Average stars ≥ 4? Output: 1/0/-1") 806
807 (4)=LLM("Results: [{(0)}, {(1)}, 807
808 {(2)}, {(3)}]. 808
809 AND aggregation. Output final 809
810 score.") 810

```

Adaptation Output (if attacks detected).

```

811 Detected: Attack patterns in reviews 812
813 [3, 7] 813
814 Modification: Exclude reviews [3, 7] 814
815 from steps (1), (3) 815

```

Execution Trace.

```

816 Step 0: PriceRange=2 → 1 (satisfied) 817
818 Step 1: Reviews mention "cozy", 818
819 "nice vibe" → 1 819
820 Step 2: Hours "11:00–20:00" contains 820
821 window → 1 821
822 Step 3: Elite avg=4.2 → 1 822
823 Step 4: AND(1,1,1,1) = min = 1 823
824 Final:  $\sigma(I, R) = +1$  (recommend) 824

```

B.6 Script Caching Strategy

Validated LWT scripts are cached per request to amortize
planning cost:

- Planning runs once when a new request type is encountered
- The compiled script is stored with a hash of the request structure
- Subsequent items with the same request reuse the cached script
- Only Adaptation and Execution run per-item ($O(k)$ calls for k conditions)

This caching reduces total LLM calls from $O(n \cdot k)$ to $O(1 + n \cdot k)$ where the planning overhead is amortized across n candidate items.

C Experiment Details

C.1 Data Construction

Restaurant Selection. We select restaurants from the Yelp Open Dataset using a multi-stage pipeline: (1) Filter by metropolitan area and target category (cafes); (2) Rank candidates by review volume to ensure sufficient evidence; (3) Apply LLM-based category verification to filter false positives. We retain 20 restaurants meeting quality thresholds.

Review Sampling. For each restaurant, we construct a document set of up to 20 reviews. To capture diverse user sentiments, we stratify sampling by star rating, targeting 4 reviews from each level (1–5 stars) where available. Within each stratum, we select reviews by descending text length to maximize information density. All reviewer identifiers are anonymized.

C.2 Evidence Types

We support four evidence types for evaluating user request conditions:

item_meta. Direct lookup of structured item attributes (e.g., `attributes.WiFi`, `attributes.NoiseLevel`). Missing values yield “unknown” (0); boolean attributes map directly to ± 1 . Categorical attributes use declarative matching rules: if `true` condition matches, return $+1$; if `false` condition matches, return -1 ; otherwise return 0.

item_meta_hours. Time-window containment verification. Given business hours (e.g., “7:00–18:00”) and required range (e.g., “10:00–15:00”), returns $+1$ if the required range falls entirely within business hours, -1 otherwise.

review_text. LLM-based review analysis. An LLM judges each review independently for the target aspect, returning $+1$ (positive mention), 0 (neutral/not mentioned), or -1 (negative mention). Judgments are cached for reproducibility. The final result uses *symmetric consensus voting* with a 75% threshold:

$$\text{result} = \begin{cases} +1 & \text{if } \frac{n_+}{n_+ + n_-} \geq 0.75 \\ -1 & \text{if } \frac{n_-}{n_+ + n_-} \geq 0.75 \\ 0 & \text{otherwise (no consensus)} \end{cases} \quad (5)$$

review_meta. Reviewer and review-level metadata conditions. Supports filtering (e.g., elite reviewers only, reviews from 2018+, minimum star rating) and weighted aggregation (by review count, usefulness votes, or social connections).

C.3 Request Structure

Each request is represented as a Boolean expression tree. Leaf nodes specify individual conditions with aspect names and evidence specifications. Internal nodes apply Boolean operators (AND/OR) with three-valued semantics:

AND	+1	0	-1	OR	+1	0	-1
+1	+1	0	-1	+1	+1	+1	+1
0	0	0	-1	0	+1	0	0
-1	-1	-1	-1	-1	+1	0	-1

C.4 Request Groups

G01 (R00–R04): Flat / Simple Metadata. Five requests using only direct item attributes with flat AND composition. Example: “Looking for a quiet cafe with free Wi-Fi and no TV.”

G02 (R05–R09): Mixed / Item Meta + Review Text. Five requests combining structured metadata filters with unstructured review text analysis. Example: “Budget-friendly indoor cafe where reviews mention good matcha.”

G03 (R10–R14): Flat / Computed Metadata. Five requests requiring hours interpretation. Example: “Quiet cafe with free Wi-Fi open Monday 10am–3pm.”

G04 (R15–R19): Flat / Social Metadata. Five requests using reviewer credibility signals. Example: “Places where experienced Yelp reviewers gave high ratings.”

C.5 Ground Truth and Evaluation

Label Generation. Ground truth labels are precomputed deterministically. For each (restaurant, request) pair: (1) Evaluate each condition against available evidence; (2) Aggregate results using the request’s Boolean structure; (3) Store the final label ($+1$, 0, or -1) with full evidence breakdown. LLM judgments for review text are cached to ensure reproducibility across runs. The resulting dataset contains 400 labeled pairs (20 restaurants $\times$ 20 requests).

Top-1 Selection. Each request is designed to have exactly one restaurant with a $+1$ (recommend) ground truth label. This enables unambiguous evaluation: for each request, there is a single correct answer among the 20 candidates.

Hits@k Metric. We evaluate recommendation accuracy using Hits@k:

$$\text{Hits@k} = \frac{1}{|R|} \sum_{r \in R} \mathbf{1}[\text{rank}(r) \leq k] \quad (6)$$

where R is the set of requests and $\text{rank}(r)$ is the position of the ground-truth positive item in the method’s ranking for request r . Hits@1 measures whether the method’s top recommendation exactly matches the gold item.

C.6 Baselines

We evaluate against 13 zero-shot prompting baselines organized into two categories.

Reasoning-Based Methods

Chain-of-Thought (CoT). Zero-shot prompting where the LLM analyzes all candidates in a single context window and selects the best match. Given a user request and the set of 20 candidate restaurants (each with metadata and reviews), the LLM is prompted to: (1) analyze each restaurant’s suitability for the user’s stated requirements; (2) select the single best-matching restaurant from the candidates. No few-shot examples are provided, testing the model’s inherent reasoning capabilities.

Self-Consistency (CoT-SC). Extends Chain-of-Thought by sampling multiple independent reasoning paths (typically 5) and selecting the final answer via majority vote. This reduces sensitivity to any single reasoning chain’s errors.

933 **ReAct.** Interleaves reasoning (Thought) with actions (ex-
934 amine reviews, check ratings, analyze requirements) in a
935 multi-turn loop. The model alternates between thinking about
936 the problem and taking actions to gather information, with a
937 maximum of 5 steps before making the final selection.

938 **Least-to-Most (L2M).** Decomposes the user request into
939 a sequence of simpler subproblems, solves each subproblem
940 incrementally (with solutions to earlier subproblems available
941 as context), then synthesizes the final recommendation from
942 all partial solutions.

943 **Plan-and-Solve (PS).** Generates an explicit multi-step plan
944 before execution. The model first devises a strategy for eval-
945 uating candidates, then systematically follows the plan to an-
946 alyze each restaurant against the user’s requirements.

947 **Self-Ask.** Generates follow-up questions about the user re-
948 quest (e.g., “What specific attributes should I check for this
949 requirement?”), answers each question using the available ev-
950 idence, then uses the accumulated answers to make the final
951 selection.

952 **Decomposed Prompting.** Uses specialized sub-task han-
953 dlers orchestrated by a decomposer. Handlers include: ex-
954 tract_requirements (parse user constraints), analyze_reviews
955 (assess review sentiment), count_evidence (tally support-
956 ing/opposing mentions), and make_decision (synthesize final
957 ranking). Each handler focuses on a specific capability, im-
958 proving modularity.

959 **Ranking-Based Methods**

960 **Pointwise (Fine-Grained).** Scores each candidate indepen-
961 dently on a 4-point relevance scale: Highly Relevant (3), Rel-
962 evant (2), Somewhat Relevant (1), Not Relevant (0). Can-
963 didates are then ranked by their scores. Complexity: $O(n)$
964 LLM calls for n candidates.

965 **PaRaDe.** Few-shot demonstration-based ranking that pro-
966 vides explicit examples of how to compare and rank candi-
967 dates. The demonstrations guide the model toward consistent
968 ranking behavior.

969 **Pairwise (PRP).** Compares all pairs of candidates with the
970 prompt “Which is more relevant to the user’s request: A or
971 B?” Aggregates pairwise wins into a total score for each can-
972 didate, then ranks by win count. Complexity: $O(n^2)$ LLM
973 calls for n candidates.

974 **RankGPT.** Sliding-window listwise reranking that pro-
975 cesses candidates in overlapping windows. The model ranks
976 items within each window, and the process iterates to refine
977 the global ranking. Balances context length constraints with
978 ranking quality.

979 **Setwise.** Compares candidates in sets (typically 3–5 items)
980 rather than pairs. For each set, the model identifies the best
981 candidate; this process repeats with tournament-style elimi-
982 nation. Offers a middle ground between pairwise granularity
983 and listwise efficiency.

984 **Listwise.** Single-pass ranking where the LLM outputs a
985 complete permutation of all candidates in one response. The
986 model sees all items simultaneously and produces the full
987 ranking directly. Complexity: $O(1)$ LLM calls (single call),
988 but requires sufficient context length.